

# C언어 자료구조 4종

Linked List · Stack/Queue · Binary Tree · BST

여기서는 그 함수를 쓰기 전에 노드가 어떻게 생겼는지, 화살표가 어디로 가는지만 먼저 고정한다.

| 구조          | 한 줄                    | 핵심 포인터                |
|-------------|------------------------|-----------------------|
| Linked List | 칸이 다음 칸 주소를 들고 한 줄로 선다 | head, cur, prev, next |
| Stack       | 맨 위만 넣고 뺀다 (LIFO)      | top                   |
| Queue       | 앞은 빼고 뒤에 넣는다 (FIFO)    | front, rear           |
| Binary Tree | 한 칸이 왼쪽·오른쪽 자식 최대 2개   | left, right           |
| BST         | 트리 + 왼쪽은 작고 오른쪽은 크다    | root, parent, cur     |

추천 순서: 구조체+포인터 → Linked List → Stack/Queue → Binary Tree → BST. 트리 문제는 리스트를 두 갈래로 나눈 것과 같다.

## 0. 네 구조가 같이 쓰는 재료

배열은 칸이 붙어 있어서  $a[i+1]$ 로 다음 값을 찾는다. 자료구조 네 종류는 칸이 떨어져 있고, 대신 “다음 칸 주소”를 변수에 적어 둔다. 그 주소 변수가 포인터다.

```
typedef struct Node {
    int item;        /* 실제 값 */
    struct Node *next; /* 다음 칸의 주소. 없으면 NULL */
} Node;
```

```
Node *head = NULL; /* 아직 아무 칸도 없음 */
head = malloc(sizeof(Node));
head->item = 10;
head->next = NULL;
```

변수는 상자를 들고 있지 않다. 상자의 주소만 들고 있다.



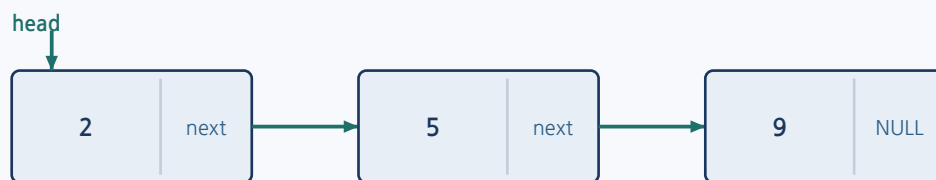
head가 상자를 들고 있는 것이 아니라, 상자 위치만 적힌 쪽지를 들고 있다.

## 포인터 기호

| 기호                | 읽는 법                      |
|-------------------|---------------------------|
| <b>p</b>          | 주소가 적힌 쪽지                 |
| <b>*p</b>         | 그 주소에 있는 값                |
| <b>p-&gt;item</b> | (*p).item 과 같다. 칸 안의 item |
| <b>p-&gt;next</b> | 다음 칸으로 가는 쪽지              |
| <b>NULL</b>       | 가리키는 칸이 없음. 여기서 멈추라는 표시   |

## 1. Linked List

한 줄로 선 사람들이다. 각 사람은 자기 번호(item)와 “다음 사람 주소”(next)만 안다. 줄의 맨 앞 주소를 들고 있는 변수가 head다.



한 줄로 선 사람들. 각자 다음 사람 주소만 안다.

head → 2 → 5 → 9 → NULL

## 문제를 풀 때 항상 그리는 세 포인터

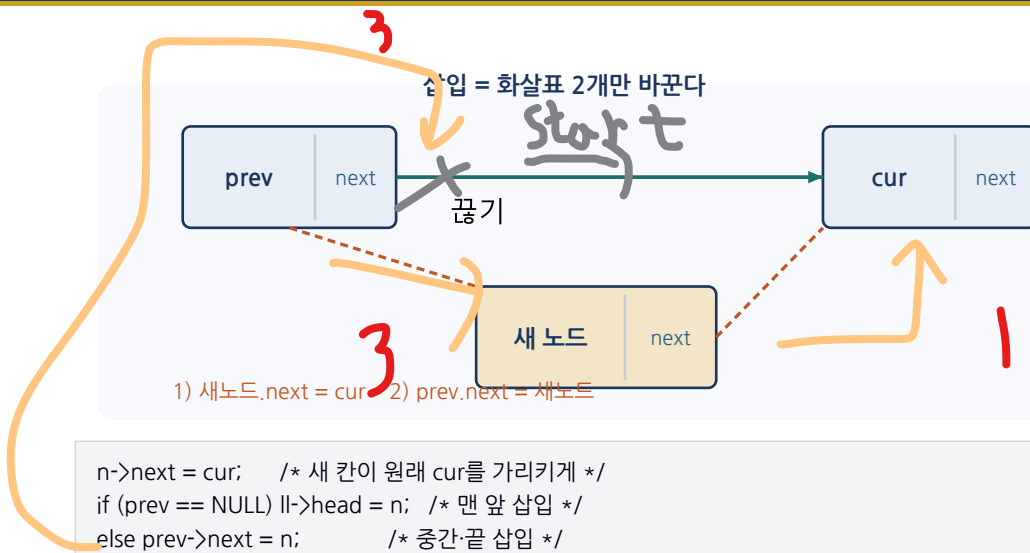
| 이름          | 역할                                      |
|-------------|-----------------------------------------|
| <b>head</b> | 줄의 시작. 첫 칸을 바꾸면 head도 바뀌어야 한다           |
| <b>cur</b>  | 지금 보고 있는 칸. cur = cur->next 로 한 칸 이동    |
| <b>prev</b> | 바로 앞 칸. 삽입·삭제할 때 앞사람의 next를 고쳐야 해서 필요하다 |

```

ListNode *cur = ll->head;
ListNode *prev = NULL;
while (cur != NULL) {
    /* cur->item 을 본다 */
    prev = cur;
    cur = cur->next;
}
  
```

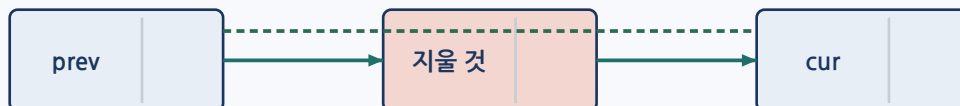
## 삽입

새 칸을 prev와 cur 사이에 끼운다. 고치는 화살표는 두 개뿐이다.



## 삭제

삭제 = prev.next를 지울 노드 다음으로



그 다음 free(지울 노드). 빈 리스트/첫 노드/마지막 노드를 따로 생각한다.

```

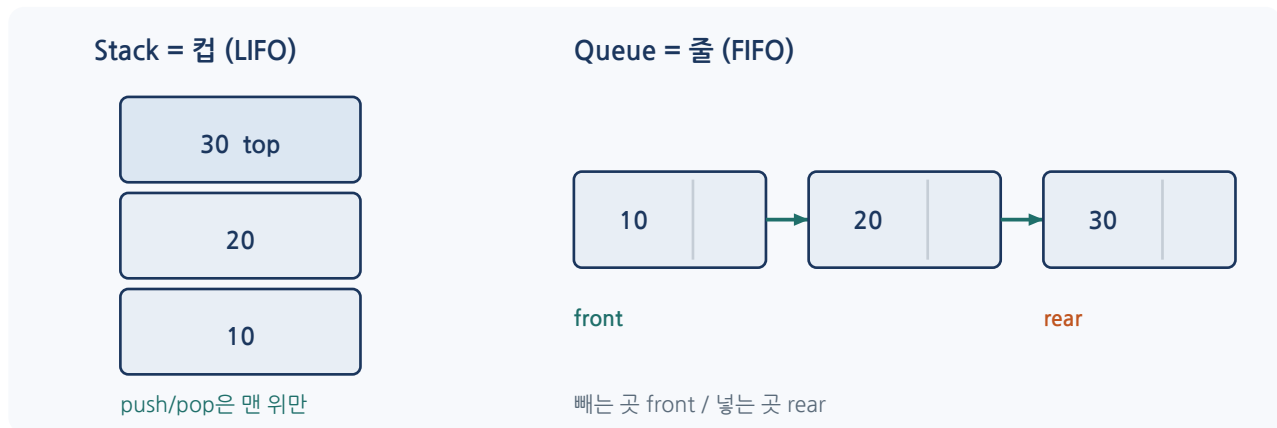
if (prev == NULL) ll->head = cur->next;
else prev->next = cur->next;
free(cur);

```

반드시 따로 생각하는 네 경우: 빈 리스트(head==NULL), 첫 칸 삭제, 마지막 칸 삭제, 칸이 하나뿐일 때

## 2. Stack과 Queue

둘 다 리스트와 같은 칸을 쓴다. 다른 점은 “어디서만 넣고 빼느냐”다. Stack은 한쪽 끝만, Queue는 한쪽에서 빼고 다른 쪽에서 넣는다.



|       | Stack                     | Queue                         |
|-------|---------------------------|-------------------------------|
| 넣는 함수 | <code>push</code> (top 위) | <code>enqueue</code> (rear 뒤) |
| 빼는 함수 | <code>pop</code> (top)    | <code>dequeue</code> (front)  |
| 엿보기   | <code>peek / top</code>   | <code>front</code> 값          |
| 비었나   | <code>isEmptyStack</code> | <code>isEmptyQueue</code>     |
| 비유    | 쌓인 접시, Ctrl+Z             | 은행 줄, 프린터                     |

학생이 쓸 코드는 이 네 함수를 반복해서 호출하는 것이

대부분이다.

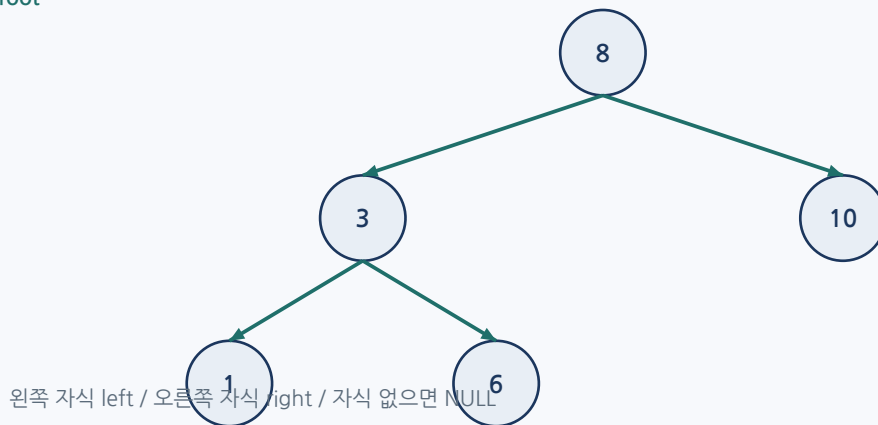
```
push(s, 10);
x = pop(s);    /* 가장 최근에 넣은 값이 나온다 */
enqueue(q, 10);
y = dequeue(q); /* 가장 먼저 넣은 값이 나온다 */
```

### 3. Binary Tree

리스트는 next가 하나라 줄이다. 트리는 left와 right 두 갈래다. 한 노드의 자식은 최대 두 명이다. 맨 위가 root, 자식이 없는 칸이 leaf다.

```
typedef struct TreeNode {
    int item;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;
```

root



#### 재귀 한 덩어리만 외우면 트리 문제 대부분이 풀린다

```
void walk(TreeNode *n) {
    if (n == NULL) return; /* 1. 바닥: 칸이 없으면 끝 */
    /* 2. 지금 칸에서 할 일 */
    walk(n->left); /* 3. 왼쪽 작은 트리 */
    walk(n->right); /* 4. 오른쪽 작은 트리 */
}
```

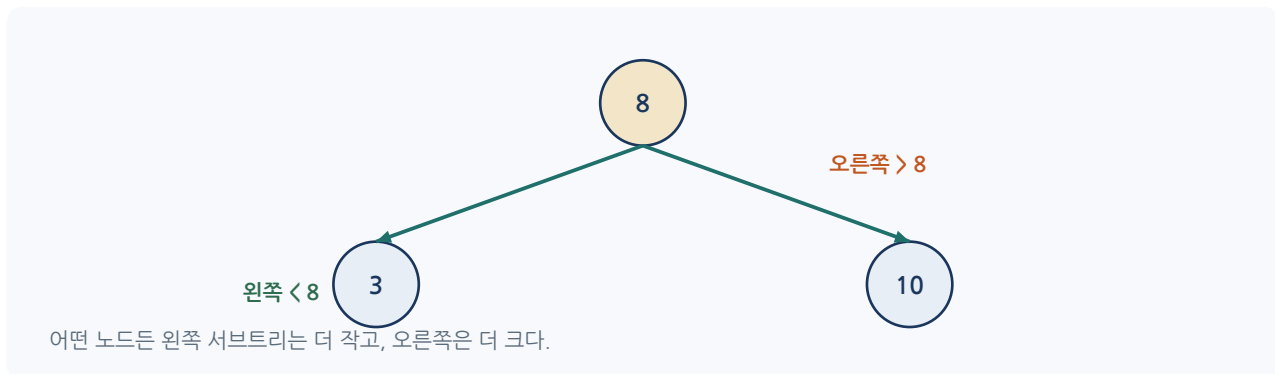
세 가지 순회는 “지금 칸을 언제 출력하느냐”만 다르다.

| 이름           | 순서        | 그림의 8-3-1-6-10 |
|--------------|-----------|----------------|
| preorder 전위  | 나 → 왼 → 오 | 8, 3, 1, 6, 10 |
| inorder 중위   | 왼 → 나 → 오 | 1, 3, 6, 8, 10 |
| postorder 후위 | 왼 → 오 → 나 | 1, 6, 3, 10, 8 |

높이(height)는 “이 칸에서 앞까지 가장 긴 길”이다. 빈 트리는 -1 또는 0으로 정하는 과제 규칙을 따른다. 보통  $\max(\text{왼쪽높이}, \text{오른쪽높이}) + 1$  이다.

## 4. Binary Search Tree

모양은 Binary Tree와 같다. 규칙 하나만 더 있다. 어떤 칸을 기준으로 왼쪽은 모두 더 작고, 오른쪽은 모두 더 크다. 그래서 찾는 값이 지금 칸보다 작으면 왼쪽만 보면 된다.



### 삽입

```
if (*node == NULL) { /* 빈자리에 새 칸 */
    *node = 새노드; return;
}
if (value < (*node)->item)
    insert(&(*node)->left, value); /* 작은 값은 왼쪽 */
else if (value > (*node)->item)
    insert(&(*node)->right, value); /* 큰 값은 오른쪽 */
/* 같은 값은 과제 규칙을 따른다. 보통 무시하거나 오른쪽에 둔다 */
```

의미: 빈자리에 도착해서 새 상자를 생성하는 코드  
화살표 매핑: 상단 트리 그림에서 3의 왼쪽 아래나 10의 자식 방향에 생기는 "빈 곳(NULL)"을 가리킵니다.

### 삭제 세 경우

의미: 추가하려는 값이 현재 값보다 큰 경우  
화살표 매핑: 상단 트리 그림의 8 -> 10 방향 (오른쪽 화살표)을 가리킵니다. (오른쪽 > 8)

화살표: 나를 건너뛰고 부모 노드의 화살표를 내 자식 노드로 직접 바로 연결합니다

화살표: 오른쪽 서브트리의 가장 왼쪽 아래 노드 값을 찾아 지울 노드 위치로 값을 복사해 오는 화살표를 가시킵니다

화살표: 부모 노드가 나를 가리키던 기존 화살표를 끊고 NULL로 바꿉니다.

자식 0개  
그냥 지우고  
부모 링크를 NULL

자식 1개  
부모를 그 자식에게 바로 연결

자식 2개  
오른쪽에서 가장 작은 값을 복사한 뒤 그 노드 삭제

자식이 둘이면 "오른쪽 서브트리에서 가장 작은 칸"(왼쪽을 계속 따라간 끝)의 값을 지울 칸에 복사하고, 그 작은 칸을 삭제한다. 그 칸은 왼쪽 자식이 없으므로 경우 0 또는 1이 된다.

### 레벨 순회는 큐를 쓴다

한 층을 왼쪽에서 오른쪽으로 보려면 큐가 필요하다. root를 enqueue하고, 하나 꺼내 출력한 뒤 그 자식들을 enqueue한다

```
enqueue(&q, root);
while (!비었음) {
    n = dequeue(&q);
    printf("%d ", n->item);
    if (n->left) enqueue(&q, n->left);
    if (n->right) enqueue(&q, n->right);
}
```

ex : while (!is\_empty(&q))

enqueue(&q, root);: 맨 위 뿌리 노드(root)를 큐에 줄 세웁니다.

n = dequeue(&q);: 줄의 가장 맨 앞에 있는 노드 하나를 꺼냅니다.

printf("%d ", n->item);: 꺼낸 노드의 숫자를 출력합니다.

if (n->left)... / if (n->right)...: 꺼낸 노드의 왼쪽/오른쪽 자식 노드가 있다면, 다음 차례로 줄을 세웁니다(enqueue).

## 문제를 열었을 때 할 일

README가 시키는 순서 그대로다. 답을 외우지 말고 그림부터 그린다.

| 순서 | 행동                                                  |
|----|-----------------------------------------------------|
| 1  | 문제 PDF에서 “무엇을 반환/출력하는지” 한 줄로 적는다                    |
| 2  | 해당 Q*.c를 연다. 이미 있는 함수는 건드리지 않는다                     |
| 3  | 빈 함수 안에서 노드 2~3개짜리 그림을 연습장에 그린다                     |
| 4  | head/cur/prev 또는 left/right가 함수 시작 때 어디를 가리키는지 표시한다 |
| 5  | 한 칸 이동이 cur=cur->next 인지 n=n->left 인지 정한다           |
| 6  | 빈 경우(NULL)를 먼저 return 한다                            |
| 7  | 컴파일한 뒤 작은 입력 2~3개로 출력이 그림과 같은지 본다                   |

## 자주 막히는 지점

| 증상                | 원인                                    |
|-------------------|---------------------------------------|
| 컴파일은 되는데 바로 죽음    | *NULL 접근. cur==NULL인지 먼저 검사           |
| 첫 칸만 이상하다         | head를 안 고쳤다. prev==NULL 경우를 빠뜨림       |
| 무한 루프             | next를 안 전진했거나, 환형인데 NULL을 기다림         |
| 트리 값이 전부 안 나옴     | left만 가고 right를 안 감. 재귀 두 갈래를 둘 다 호출  |
| BST 삽입 후 순서가 이상   | 작은데 오른쪽으로 보냄. 비교 방향을 그림과 대조           |
| 스택으로 큐를 뒤집었더니 그대로 | enqueue/push를 반대로 씀. LIFO/FIFO를 다시 그림 |

기본 함수(insertNode, removeNode, push, pop, enqueue, dequeue, insertBSTNode)는 이미 채워져 있다. 빈 칸 함수만 채우면 된다. 직접 next를 조작하기 겁나면, 리스트 문제는 findNode + insertNode + removeNode 조합으로도 대부분 구현할 수 있다.